Leveraging LLM Reflection to Improve Small Language Model Agents' Capabilities

Aissa Hadj Mohamed[1][0000−0001−7354−7240], Leandro A. Villas[1][0000−0002−3372−3366], and Julio Cesar dos Reis[1][0000−0002−9545−2098]

Universidade Estadual de Campinas (UNICAMP), Campinas SP , Brazil
a265189@dac.unicamp.br, {lvillas, jreis}@ic.unicamp.br

Abstract. In recent years, the emergence of Small Language Models (SLMs) has opened new possibilities for deploying lightweight, efficient AI systems across various applications. However, SLMs produce limited outputs for tasks requiring complex reasoning and self-evaluation. Their limited capacity often results in suboptimal effectiveness, particularly in user-aligned generation tasks. This study introduces a novel framework that leverages Large Language Models (LLMs) to provide reflective feedback on the outputs generated by SLM-based autonomous agents. Given a user prompt, the SLM agent produces an initial response, which the LLM then evaluates in the context of user feedback to produce a reflection. This reflection is stored in a dynamic memory module for reflection. This provides a growing repository of corrective insights tailored to the SLM's historical operation. These reflections are used to guide the SLM agent in outputting improved upcoming responses to user prompts. Our experimental evaluations, conducted on the SLMs Deepseek-8B and Phi-2 with 8 billion and 2.7 billion parameters, respectively, on the ARC challenge dataset, demonstrate improvements in output quality compared to the baseline approach, where the SLM agents self-reflect on their outputs. When prompted with external reflections, Deepseek-8B increases its accuracy score from 62.78% to 81.38% versus 77.70% with its self-reflections. Our findings highlight the effectiveness of externalized reflection memory as an augmentation strategy to enhance SLM agents' outcomes without increasing inference-time cost.

Keywords: Intelligent Agents · Large Language Model · SLM Agent · Memory Reflection.

1 Introduction

Large Language Models (LLMs) are reshaping Human-Computer Interaction by enabling a new wave of intelligent systems—such as ChatGPT[1], Gemini[2],

[1] ChatGPT: Conversational AI Model, https://openai.com/chatgpt
[2] Gemini: A Family of Highly Capable Multimodal Models, https://deepmind.google/gemini

Claude[3], and DeepSeek[4], that are capable of generating coherent, context-aware, and goal-oriented content. Their integration into digital services, especially in customer support, promises to yield measurable gains in efficiency, cost reduction, and user satisfaction, as users increasingly view these interactions as intuitive and effective sources of information [10]. LLMs are now being applied to various generative and reasoning-intensive tasks such as content creation, code and music generation, personalized assistance, multilingual translation, and open-domain question answering ([2], [16]), underlining their versatility and growing importance in the design of intelligent systems.

Despite their capabilities, LLM-powered applications face limitations that constrain their effectiveness in complex and/or dynamic scenarios. A significant challenge is hallucination, a phenomenon in which the model produces incorrect or fabricated information [14]. LLMs are inherently stateless, lacking persistent memory of prior interactions.

The memory modules can be integrated into LLMs to form agent architectures capable of perceiving and retaining information from their environment. The memory module stores, processes, and retrieves task-relevant information. These memory-augmented conversational agents leverage accumulated experiences to inform future decisions, thus enabling more consistent, coherent, and contextually grounded behavior [14].

Building on this principle, the Retrieval-Augmented Planning (RAP) framework proposed in [3] introduces a mechanism to dynamically retrieve and apply relevant past experiences, thus enhancing the agent's ability to plan and act in alignment with the situational context. Memory enables the agent to retain historical experiences and support decision-making. Rather than requiring retraining to acquire new or previously unseen information, the memory allows the agent to accumulate knowledge dynamically, enhancing adaptability while reducing computational costs [13].

The memory reflection is one type of memory that allows the LLM agent to learn from past experiences and improve its future actions. In [13], the authors introduced the concept of "Reflexion" as a technique that leverages verbal reinforcement to help LLM-based agents learn from past mistakes. This approach is an alternative to traditional trial-and-error learning, as conventional reinforcement learning methods often require large amounts of training data and expensive model fine-tuning.

Within the Reflexion framework, an LLM agent is prompted to self-evaluate its past actions and generate self-critical outputs. Given a user prompt, the agent's initial response, and feedback from the environment (*e.g.*, user corrections or external validation), the agent analyzes its previous answer to identify flaws in reasoning, factual inaccuracies, or logical inconsistencies. It then formulates strategies for improvement, refining its approach when faced with the same or similar queries. This iterative reflection process enables the agent to continually

[3] Claude: Conversational AI Model, https://www.anthropic.com
[4] DeepSeek, https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B

improve the quality, accuracy, and reliability of its responses until a satisfactory level of performance is achieved.

Reflection memory offers several key benefits for enhancing the effectiveness and trustworthiness of LLM agents. By revisiting and evaluating their prior outputs, LLMs can improve the accuracy and quality of their responses through self-correction. This reflective process helps identify and mitigate biases, promoting more balanced and fair outcomes. Reflection memory contributes to increased transparency by shedding light on the agent's decision-making process, enabling users to understand better and trust the rationale behind generated responses. These advantages make reflection memory a powerful mechanism for supporting continual learning, interpretability, and robustness in LLM-based systems.

When computational efficiency, scalability, and deployment cost are critical factors, Small Language Models (SLMs) could be an alternative to LLMs. Due to their significantly reduced parameter count, SLMs require far less computer memory and processing power, making them ideal for running on edge devices, mobile platforms, or resource-constrained environments.

The reflection memory benefits in LLMs do not naturally extend to SLM agents because of their limited architectural capacity. Unlike large-scale models, SLMs struggle to perform effective self-evaluation or maintain complex memory structures, constraining their ability to reflect meaningfully on past outputs. Their reduced parameter count makes it more difficult for SLM agents to identify subtle errors or biases. Moreover, challenges such as self-assessment bias and overfitting to prior experiences are exacerbated in SLMs, often preventing them from leveraging reflection memory to its full potential. As a result, designing effective reflection mechanisms for SLM agents remains an open and pressing research challenge ([1], [6], [9], [16]).

This study proposes a reflective framework designed to enhance the quality of reflection memory and improve agent effectiveness. Upon receiving a user prompt, an SLM agent generates an initial answer. The user then provides feedback based on this response. The user prompt, the SLM's output, and the feedback are concatenated and submitted to a Large Language Model (LLM), which is tasked with generating a reflection. This reflection is stored in the SLM agent's memory and used to guide SLM's future responses. By incorporating LLM-generated critique grounded in external feedback, the SLM agent accumulates a history of reflections, enabling it to improve over time through memory-based adaptation.

Our experimental evaluations on the ARC Challenge dataset demonstrate improvements in output quality compared to the baseline approach, where the SLM agent self-reflects on its outputs. Our results highlight the effectiveness of externalized reflection memory as an augmentation strategy to enhance SLM outcome quality without increasing inference-time cost. Our externally-generated reflections approach results in (i) more efficient memory usage, (ii) more effective prompting under limited context window sizes, and (iii) faster retrieval, im-

proving scalability in memory-augmented prompting. All experimental results reported in this study are available at our public repository[5].

This study provides the following key contributions:

- Propose a novel reflection-based framework that leverages LLM-generated introspection to enhance SLM results.
- Introduce a collaborative setup where an LLM, informed by user feedback, produces structured reflections stored in a memory module for future reuse.
- Experimentally demonstrate that external LLM reflections lead to higher-quality outputs compared to SLM self-reflection.
- Provide a detailed analysis of reflection length and content, showing that LLM-generated reflections are more concise and semantically compelling than SLM ones.

The remaining of the article is structured as follows. Section 2 presents relevant existing solutions in the literature. Section 3 presents our designed and implemented solution. Section 4 presents the experimental methodology conducted to assess our external-reflection approach. Section 5 reports on the experimental results. Section 6 discusses our findings and points out future investigations. Section 7 wraps up the conclusion remarks.

2 Related Work

This section presents recent literature solutions that tackle the challenges related to the reflection memory component in language model-based agents. We discuss a few studies that explore the use of LLMs to enhance the effectiveness of SLM agents.

Renze *et al.* [12] investigated how incorporating reflection memory may enhance LLM agent performance. Agents equipped with distinct self-reflection strategies, such as retrying after failure, receiving general advice, or analyzing the cause of an error, consistently outperformed a baseline that did not include reflection. Their findings suggested that enabling agents to reflect on mistakes using environmental feedback helps reduce repeated errors and prevents unproductive behavior loops.

Zhao *et al.* [17] criticized that advanced language models, such as GPT-4 and Claude, are primarily accessible via APIs, with their underlying parameters remaining proprietary. This underscores the need for methods that enable learning from experience without parametric updates. The authors proposed the ExpeL agent, which autonomously accumulates knowledge from training tasks and leverages past experiences during inference. Their experiments showed that ExpeL consistently improves results as more experiences are gathered.

Wu *et al.* [15] highlighted a core limitation of current LLMs: their token generation depends primarily on input length rather than the complexity of the reasoning task. This often results in overconfident, logically coherent yet

[5] https://github.com/aissahm/slm-reflect

incorrect outputs that compound earlier mistakes, a phenomenon described as the "snowball effect". Their study proposed the Editable Large Language Model (E-LLM), which introduces flexible editing capabilities such as token insertion and deletion. This allows the model to self-correct during generation, adapt reasoning effort to task difficulty, and improve output consistency without relying on predefined reasoning chains.

LLMs face challenges such as continuous decision-making, lack of long-term memory, and limited context windows in dynamic environments [7]. A significant amount of research has focused on enhancing the ability of LLMs to improve themselves. Some researchers are investigating the use of carefully crafted prompts to help models learn how to improve, although this approach typically only works for one-off tasks. Others are tweaking how models receive feedback during tasks, which helps them improve at thinking things through [7].

To address challenges in multi-tasking and long-span reasoning, Liang *et al.* [7] proposed the SAGE framework (Self-evolving Agents with Reflective and Memory-augmented Abilities), which consists of three components: a User, an Assistant, and a Checker agent. SAGE integrates iterative feedback, reflection, and memory optimization guided by the Ebbinghaus forgetting curve to enhance adaptability and reduce cognitive load. Its MemorySyntax mechanism ensures that only essential information is retained. Experimental results demonstrated that SAGE yields substantial performance gains—up to 2.26× on closed-source models and 57.7%–100% on open-source models, particularly benefiting smaller models. Notably, SAGE demonstrated that LLMs can improve over time through self-reflection without additional training.

In Lippmann *et al.* [8], the authors observed that LLM agents with memory reflection often underperform when initially successful or when using smaller models. This is because traditional reflection methods focus solely on failures, neglecting the value of reinforcing successful behaviors. They introduced Sweet&Sour, a framework incorporating positive (sweet) and negative (sour) experiences into the reflection process. This allows agents to reinforce effective strategies while still learning from mistakes. When rewards are achieved, the agent is prompted to articulate and generalize the success factors. Sweet&Sour employs a dual-buffer memory system, short-term and long-term, organized by outcome and recency, to manage and retrieve reflections efficiently.

The study conducted by Li *et al.* [5] designed a solution where an LLM model generates various candidate responses based on a prompt. Then, given a set of response candidates and the prompt, the same LLM model outputs a final answer. Given an input X to the LLM model, this approach contains three steps: (1) *Exploration*: Given an input X, prompt LLM M to generate K candidate responses r_j, (2) *Reflection*: For each response r_j, prompt M with the concatenated input $[X; r_j]$ to generate a self-critique c_j, and (3) *Revision*: concatenate the K response-reflection pairs into a new input and prompt M to generate an improved output.

Other recent studies have focused on improving the effectiveness of Small language models (SLMs) in executing tasks. According to Kang *et al.* [4], SLMs

often fail to replicate LLM-level performance, especially on tasks requiring rare knowledge or computation, despite chain-of-thought (CoT) distillation. These SLMs hallucinate or make critical errors when facing novel, multi-step, or tool-augmented tasks. Kang *et al.* [4] introduced Agent Distillation, a framework that distills not just reasoning, but full agentic behavior from large LLM agents into smaller models. Their work aimed to teach SLMs to emulate agentic behavior (such as tool use and reasoning) by distilling interactive trajectories (reason-act-observe) generated by an LLM (the teacher model). Their study showed that SLMs (with as low as 0.5B parameters) match or outperform larger CoT-distilled models (up to 7B) across eight distinct tasks (factual + mathematical), and improve outcome quality on complex reasoning tasks.

According to Piao *et al.* [11], most reasoning distillation methods simply fine-tune SLMs on teacher-generated Chain-of-Thought (CoT) reasoning. This often results in surface-level mimicry rather than true reasoning capability, since SLMs may fail to internalize the underlying knowledge. Piao *et al.* [11] introduced *TinyThinker*, a novel framework aimed at enhancing reasoning in SLMs by internalizing knowledge progressively and refining it through self-reflection. The goal of *TinyThinker* is to internalize reasoning in SLMs via structured CoT + self-reflection. The inputs for reflection are self-generated reasoning steps. The LLM is used only in the teacher role to generate initial CoT data. The authors demonstrated that *TinyThinker* outperforms traditional fine-tuning and some recent CoT-based distillation approaches on OpenBookQA and StrategyQA datasets. *TinyThinker* is also strong at internalizing reasoning rather than just reproducing patterns.

While existing studies have adopted a similar approach to ours by using an LLM to assist an SLM agent, to the best of our knowledge, no work has investigated the effectiveness of external reflection provided by an LLM aiding the reflection of SLM agents.

3 SLM Agent Guided by LLM Reflections

This section presents our proposal for an LLM-enhanced reflection memory for Small Language Model (SLM) agent systems to enhance agent effectiveness.

3.1 Memory Reflection

Memory reflection is a component of LLM agents that enables them to reason about and learn from their own outputs, thereby improving performance through introspective analysis. In contrast to traditional learning paradigms, which often depend on external supervision or static datasets, reflection allows LLM agents to act as their own critics, identifying reasoning flaws, revisiting prior decisions, and updating behavioral strategies accordingly. The effectiveness of a reflection memory is measured by its ability to help the agent generalize and refine its responses in similar contexts.

Formally, memory reflection operates through an internal loop of self-evaluation, where an agent evaluates its past results, based on external feedback or outcome-based signals, and stores the self-critiques in memory for future use. These reflective memories guide decision-making in subsequent tasks. The reflection memory can take diverse forms, such as explicit textual summaries of errors, reformulated strategies, or abstract representations of failed reasoning paths. When integrated into new prompts, this mechanism enables LLM agents to exhibit continual learning, error correction, and adaptive planning without additional gradient-based training. Mathematically, the agent's reflection memory can be defined as a function of the initial user prompt, the agent's generated response, and the reflection prompt provided to the agent.

3.2 Our Approach to Reflective SML Agents

Applying reflection memory to SLMs presents significant challenges. Due to their limited parameter count and restricted reasoning capacity, SLMs cannot perform nuanced self-evaluation or generate sophisticated memory reflections. As a result, they struggle to identify subtle reasoning flaws, detect biases, or revise outputs based on prior experiences.

Our approach aims to improve the quality of reflection memories, as quality is defined as the extent to which a reflection memory enhances the agent's performance on subsequent prompts that are semantically similar to the one that initially triggered the reflection.

Our approach delegates the reflection process to an LLM (more capable model), which acts as a supervisory agent. Figure 1 presents our proposed approach in generating reflection memories by an LLM. Rather than relying on the SLM to self-reflect, we use the LLM to analyze the SLM's outputs, generate constructive feedback, and synthesize reflection memory on its behalf. This approach enables the SLM to benefit from the reflective capabilities of LLMs without requiring architectural modifications or fine-tuning, thus improving effectiveness through externalized introspection.

The loop begins with the user providing a prompt to an SLM agent, which generates an output (cf. Figure 1). The user or the environment provides feedback (external signal) to the agent's output. The tuple [user prompt p_u, generated answer a, external feedback f_{env}] is concatenated and submitted jointly to an LLM reflection prompt.

In the event of a negative external signal, the LLM is tasked with critiquing the SLM agent's output, evaluating the reasoning that led to its response, and questioning its prior beliefs and knowledge. In our solution, if the feedback is positive, the LLM provides a critique that reinforces the agent's strategies that lead to positive outcomes (similar to [8]'s approach).

When the SLM agent receives a new user prompt, a database function retrieves the most relevant reflections to help the agent formulate its response (cf. Subsection 3.3). This retrieval function relies on the correlation between the user prompt and the reflection embeddings.

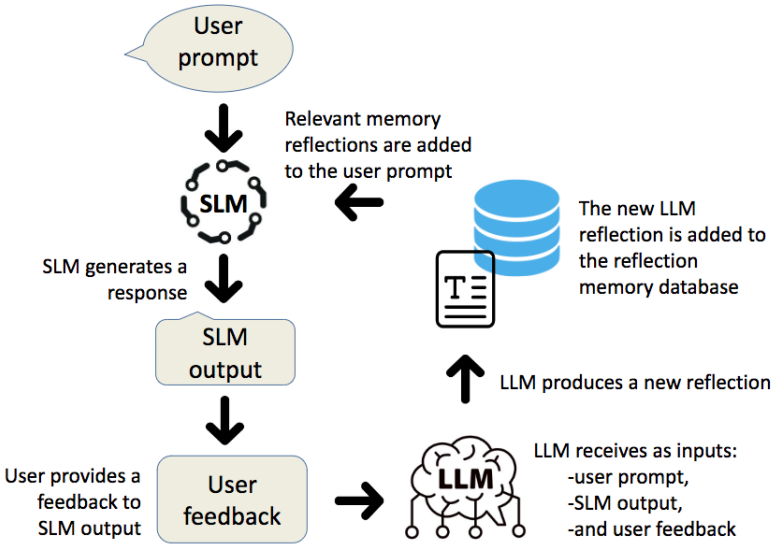

Fig. 1. In our proposed approach, an LLM provides an external reflection to an SML agent to critique its generated outputs regarding the user prompts. The external reflection is then incorporated into user prompts to guide the SLM agent in its future responses, based on its updated memory.

3.3 Reflection Memory Retrieval

Our SLM-based system stores the agent's reflection memories in a dedicated database. Each entry is a tuple of the form (user prompt, agent response, environment feedback, agent reflection). When the SLM agent receives a new user prompt, it queries its own memory to retrieve relevant past entries. The retrieval process is based on semantic similarity between the new prompt and previously stored prompts. This allows the agent to access reflections and decisions associated with similar situations in its historical experiences. Figure 2 presents the process that transforms a prompt text into its representation embeddings.

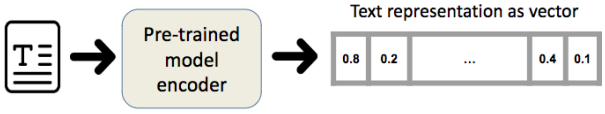

Fig. 2. Our solution uses a pre-trained encoder language model to encode the prompts. It compares the semantics between all the prompt embeddings using the cosine similarity between every pair of prompt vectors.

Our solution relies on semantic similarity between the current prompt and past prompts stored in the agent's memory to retrieve relevant reflection memories for a new user prompt. This process begins by encoding both the new user prompt and each stored past prompt into fixed-size vector representations using a pre-trained language model encoder. These text embeddings capture the semantic content of each prompt, enabling meaningful semantic comparisons.

We use the cosine similarity to retrieve the past prompts stored in the reflection memory database that are most similar to a newly submitted user prompt. More specifically, our method utilizes cosine similarity to compare the similarity between pairs of prompts. Given two vector representations $\mathbf{u}$ and $\mathbf{v}$ corresponding to two prompts, the cosine similarity between $\mathbf{u}$ and $\mathbf{v}$ is defined as:

$$\text{cosine\_sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$$

where $\mathbf{u} \cdot \mathbf{v}$ denotes the dot product of the two vectors, and $\|\mathbf{u}\|$ and $\|\mathbf{v}\|$ are their respective Euclidean norms.

The resulting similarity value ranges from -1 to 1, where 1 indicates maximum similarity, 0 indicates orthogonality (no similarity), and -1 indicates complete dissimilarity. This similarity score quantifies how closely related the new prompt is to previously encountered ones, with higher scores indicating more substantial semantic alignment (cosine similarity score close to 1.0).

The method identifies the most relevant entries by ranking the past prompts based on their similarity to the new input and retrieves the associated reflection memories. These reflections then inform the SLM agent's reasoning and adaptation process in the new situation.

4 Evaluation Methodology

This section presents the methods applied for evaluating our proposed approach and

4.1 Dataset

We selected the ARC (AI2 Reasoning Challenge) dataset [6], which consists of 1,147 multiple-choice questions requiring factual recall, scientific knowledge application, and logical inference. This makes it a valuable resource for evaluating the reasoning capabilities of LLMs. The "Challenge" contains questions that are particularly difficult for current AI models due to their need for abstract reasoning, understanding causal relationships, and synthesizing multiple pieces of information. The correct answer counts are 226 as A (22,0%), 277 as B (27,00%), 269 as C (26.0%), and 255 as D (25.0%). The correct answers are fairly distributed in the dataset. Existing LLM models with up to 7 billion parameters achieve

[6] Challenge ARC Dataset, $https://huggingface.co/datasets/allenai/ai2_arc$

accuracy scores ranging from 40% to 50% on the ARC Challenge dataset[7]. This fact helps us facilitate the analysis of experimental results.

4.2 SLMs

Our experiments explored two distinct SLMs. The first one is DeepSeek-R1-Distill-Llama-8B[8], referred to as DeepSeek-8B. This model is a distilled version of DeepSeek-R1, fine-tuned on the open-source Llama-3.1 architecture, with 8 billion parameters, making it a lightweight yet capable model suitable for evaluating reflection and reasoning under constrained computing resources.

The second SLM is Phi-2[9], referred to as Phi-2. Developed by Microsoft, Phi-2 is a compact transformer-based model with 2.7 billion parameters, specifically trained to excel in reasoning and mathematical tasks despite its relatively small size. Its training focused on high-quality, filtered data to encourage robust in-context learning, making it a strong candidate for evaluating our external-reflection proposal.

Our assessment experiments used ChatGPT-3.5[10] as the external LLM responsible for generating the reflections for both Deepseek-8b and Phi-2 execution scenarios.

4.3 Metrics

Our evaluation used three metrics to assess the effectiveness of our approach compared to the baseline scenario (SLM agent self-reflecting), as follows:

- **Accuracy score**: Accuracy was adopted as the primary evaluation metric, where higher scores indicate greater effectiveness of the SLM agents in correctly responding to user prompts.
- **Reflection length**: We measured the length of each reflection using two metrics: the number of words and the number of characters. Word count provides a high-level view of the verbosity and structure of the reflection, capturing the amount of content LLMs and SLMs generate in terms of meaningful units. Character count, on the other hand, offers a finer-grained measure that reflects the actual size of the text input, which is especially relevant for understanding memory usage and budgeting prompts. Together, these metrics enable us to evaluate the length of the reflections and their linguistic efficiency.
- **Semantic correlation with the original prompt**: We studied the semantic correlation between the reflections generated by the LLM and SLM and the ARC question. This analysis helps us assess how closely the reflections remain tied to the problem context. By measuring this alignment, we

[7] Arc Benchmark on Papers With Code, *https* : *//paperswithcode.com/sota/common − sense − reasoning − on − arc − challenge*
[8] https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B
[9] https://huggingface.co/microsoft/phi-2
[10] ChatGPT, https://chatgpt.com

aim to understand the degree of generalization or abstraction each reflection introduces. High semantic correlation indicates that the reflection is specific to the prompt, while broader reflections could offer more transferable insights. This evaluation shows how effectively simple and complex external reflections help the SLM agents in improving future reasoning performance.

– **Lexical diversity**: It refers to how varied the vocabulary is in the reflections generated by the SLMs and LLMs. The lexical diversity measures the richness or range of different words used, rather than the total number of words used. To measure the lexical diversity, we adopted the Type-Token Ratio (TTR):

$$\text{TTR} = \frac{\text{Number of unique words (types)}}{\text{Total number of words (tokens)}}$$

4.4 Procedures

Our experimental setup adopted a two-stage prompting approach to evaluate and improve the effectiveness of the SLM agent (Deepseek-8b or Phi-2) on multiple-choice science questions from the ARC Challenge dataset. This two-phase procedure aimed to isolate the impact of reflection and feedback by the LLM on answer correction.

In the first round, the SLM was prompted as an expert science question-answering agent and instructed to engage in step-by-step reasoning, analyzing all answer options and explicitly stating its final answer in the format "Answer: X". This allowed the SLM agent and the LLM to assess the model's initial reasoning process and answer selection.

We set the temperature to 0 for Deepseek-8b and Phi-2 to get deterministic answers on the ARC question-answering. We set the temperature to 0.8 for the external and self-reflections to let the LLM and SLM models creatively explore why the SLM agent's answers were incorrect in the first round of questions.

For questions the SLM answered incorrectly, we then conducted a second attempt using a revised prompt that incorporated the agent-selected option choice, the reflection, and feedback on the original response. In this second-round prompt, the model was instructed to reconsider its previous answer choice using the provided insights, but without generating any further reasoning. Instead, it was directed to select the best answer only and respond strictly in the format "My new answer is: X", reducing verbosity and increasing the likelihood of clear, decisive output. At this stage, the temperature was decreased to 0 to get deterministic answers.

We initially aimed to include the SLM agent's initial answers in the second phase, which would contain its reasoning. Nevertheless, the limited input context window size for Phi-2 (a maximum of 2048 tokens) made it impossible to include the agent's initial reasoning within the prompts. For results consistency, we removed the initial agent's answers' reasoning for Deepseek-8B.

4.5 Reflection Scenarios and Instructions

Initially, the SLM agent (DeepSeek-8B) attempts to answer all ARC questions. We prompt both the SLM and the LLM to generate reflections analyzing the SLM agent's errors for the subset of questions answered incorrectly.

The reflection prompts contain the initial ARC question, the answer generated by the SLM agent, the feedback received (correct or incorrect answer choice), and the reflection instructions. To analyze the impact of reflection quality, we introduce two prompting scenarios, simple and complex. In both reflection prompt scenarios (simple and complex), we explicitly tell the SLM agent and LLM not to solve the ARC questions, but only to evaluate the SLM-generated answers.

The **simple reflection** prompt asks the model to revisit its response briefly. The **complex reflection** prompt explicitly instructs the model to conduct a thorough analysis of its reasoning process, examining the logic behind its answer, identifying conceptual misunderstandings, and questioning its interpretation of the ARC problems. This setup enables us to evaluate the models' capacity to engage in introspective critique under varying guidance levels.

The simple reflection prompt has the following directives:

- Write a simple self-reflection to evaluate your reasoning and explain why it is wrong.
- Use simple language, and avoid complex explanations or complicated logic.
- Do not attempt to answer the question.
- Start your reflection with "I" (e.g., "I may have...").

The complex reflection prompt contains the following instructions:

- Write a detailed and analytical self-reflection that helps you reconsider your reasoning.
- Explain step-by-step why your reasoning is wrong, and how to improve next time.
- Do not attempt to answer the question.
- Start your reflection with "I" (e.g., "I may have..."), like someone self-learning.

In the second round of experiments, we focus solely on the questions that were initially answered incorrectly, ensuring consistency in the reflection task. The SLM agent is prompted with the ARC question, its initial answer choice, and the feedback received for its option selection (e.g., "Your selection choice was incorrect."), and its own reflection or the LLM's.

4.6 Experimental Settings

Table 1 briefly describes our experimental settings to evaluate our externally generated reflections approach against the baseline, where the SLM agent itself is tasked to self-reflect on its incorrect responses to the input questions.

All experimental results reported in this paper are available at our public repository: https://github.com/aissahm/slm-reflecthttps://github.com/aissahm/slm-reflect.

Table 1. Summary of Experimental Settings

Element	**Setting**	**Description**
Dataset	ARC Challenge	1,147 multiple-choice science questions requiring abstract reasoning and scientific knowledge.
Language Models	SLM: DeepSeek-8B	DeepSeek-R1-Distill model, fine-tuned on Llama-3.1 with 8 billion parameters.
	SLM: Phi-2	2.7 billion parameters, specifically trained to perform well on reasoning and mathematical tasks.
	LLM: ChatGPT-3.5	Used to generate external reflections based on user feedback and SLM responses.
Evaluation Metrics	Accuracy score	Primary metric measuring correct answer selection before and after reflection integration.
	Semantic correlation	It helps assess how closely the reflections remain tied to the problem context, or can generalize to other similar user prompts
	Reflection length	The length of each reflection using two metrics: the number of words and the number of characters.
	Lexical diversity	How varied the vocabulary is in the reflections generated by the SLM and LLM.
Prompting Procedure	First Attempt	SLM answers as a science expert using step-by-step reasoning and selecting "Answer: X".
	Second Attempt	For incorrect answers, SLM re-answers using reflection + initial option choice + feedback, selecting only "My new answer is: X" without reasoning.
Reflection Scenarios	Simple Prompt	Brief self-evaluation addressing reasoning, assumptions, and possible causes of error.
	Complex Prompt	In-depth critique of reasoning path, assumptions, and conceptual understanding with improvement suggestions.
Reflection Instructions	Source Models	Both SLM and LLM generate reflections on incorrect answers.
	Instructions	Models are explicitly told not to solve the question, only to reflect on the original response.
Temperature for LLM and SLM	First and Second Attempts	To obtain deterministic answers, we set the temperature to 0.
	Reflection phase	We set the temperature to 0.8 to let the SLM and LLM be creative in exploring the source of errors in the first attempt.

5 Experimental Results

In this section, we conduct analyses on various levels to demonstrate the effectiveness of improving the SLM agents' effectiveness, thanks to the external reflections provided by an LLM.

5.1 Accuracy analysis

Overall result analysis. Table 2 and Table 3 present the overall accuracy scores for the SLM agents Deepseek-8B and Phi-2 in the first and second attempts of question-answering. In general, for both SLM models, the reflection component helped the SLM agents to improve their overall results. For instance, Deepseek-8B increased its accuracy score from 62.78% in the first attempt to 83.39% in the second attempt with its own simple self-reflection.

Table 2. Deepseek overall accuracy scores (%) (1,147 ARC questions)

Approach	First attempt	simple reflection prompt	complex reflection prompt
DeepSeek-8B	62.78	83.39	81.82
DeepSeek-8B + ChatGPT3.5	62.78	79.13	79.21

Table 3. Phi-2 overall accuracy scores (%) (1,147 ARC questions)

Approach	First attempt	simple reflection prompt	complex reflection prompt
Phi-2	73.82	78.78	77.91
Phi-2 + ChatGPT3.5	73.82	77.13	80.69

Still analysing Table 2 and Table 3, the simple self-reflection, in the case of Deepseek, yielded the overall best performance, compared to the complex self-reflection and the external reflections (simple and complex) provided by the LLM model. Phi-2, on the other hand, performed better with the complex external-reflection provided by ChatGPT-3.5 with a performance of 80.69%.

Analysis of valid answers on the second attempt answering. Despite setting a temperature of 0 and explicitly instructing the SLM agents to select an option choice (*e.g.*, "*Only output your final answer in this format: My new answer is: X (where X is A, B, C, or D)*"), sometimes the agents did not provide a clear answer choice. Table 4 and Table 5 present results considering the

questions where the SLM agents provided a clear answer choice on the second attempt. We included the number of answers containing valid answers (correct or not) in parentheses.

Table 4. Deepseek accuracy scores considering only valid answers on second attempt (%, number of questions in parenthesis)

Approach	simple reflection prompt	complex reflection prompt
DeepSeek-8B	77.70 (303)	72.27 (305)
DeepSeek-8B + ChatGPT3.5	80.51 (231)	81.38 (231)

Table 5. Phi-2 accuracy scores considering only valid answers on second attempt (%, number of questions in parenthesis)

Approach	simple reflection prompt	complex reflection prompt
Phi2	30.15 (189)	27.16 (173)
Phi2 + ChatGPT3.5	43.67 (87)	32.37 (244)

When the answer is valid, the chances of it being correct are maximized for Deepseek-8B combined with a complex reflection provided by the LLM at an accuracy score of 81.38%. We observed that for more than 70 questions, Deepseek-8b did not use the external reflections and provide a valid answer, but provided valid ones when prompted with its own reflections (simple or complex). This suggests that we can further refine the LLM reflection prompt to help the SLM agent answer the questions more accurately.

The Phi-2, on the other hand, provides more valid answers (244 in total) when prompted with complex external reflections and yields a better performance at 43.67% with LLM's simple reflections. This indicates that Phi-2 is more capable of reasoning than Deepseek, but it is less able to learn from its own mistakes.

Accuracy analysis regarding only valid answers. For fairness, Table 6 and Table 7 present the accuracy scores when the SLM agents provided a valid answer for simple and complex reflections, provided by the LLM and generated by the SLM agents themselves. In the case of Deepseek-8B (Table 6), with a total 99 questions, the SLM agent combined with the complex LLM's reflections yields the highest accuracy score at 89.89%. The same is true for Phi-2 at 60% considering only 30 questions (Table 7).

Error loop analysis. One important benefit of reflection memory is preventing the agent from generating the same, incorrect answers (error loop). Table

Table 6. Deepseek accuracy scores considering only valid answers on second attempt (%, 99 questions)

Approach	simple reflection prompt	complex reflection prompt
DeepSeek-8B	84.84	86.86
DeepSeek-8B + ChatGPT3.5	88.88	89.89

Table 7. Phi-2 accuracy scores considering only valid answers on second attempt (%, 30 questions)

Approach	simple reflection prompt	complex reflection prompt
Phi2	40.0	40.0
Phi2 + ChatGPT3.5	43.33	60.0

8 presents this analysis regarding Deepseek-8B, which exhibits the highest error loops due to its own complex reflections. In the second attempt, in 39 out of 289 questions, Deepseek-8B failed to change its option choices despite being instructed that its initial answer choices were incorrect. The same is true for Phi-2 (cf. Table 9).

Table 8. Deepseek number of error loops (289 questions). An error loop is defined as the case where the agent selects twice the same wrong answer option.

Approach	simple reflection prompt	complex reflection prompt
DeepSeek-8B	24	39
DeepSeek-8B + ChatGPT3.5	11	17

Table 9. Phi-2 number of error loops (291 questions). An error loop is defined as the case where the agent selects twice the same wrong answer option.

Approach	simple reflection prompt	complex reflection prompt
Phi2	76	80
Phi2 + ChatGPT3.5	17	61

5.2 Reflections semantic analysis

Analysis of cosine similarities in reflection embeddings with Deepseek. Following the initial attempt on the ARC questions, we prompt both the SLM and LLM models to reflect on the incorrect responses produced by the SLM agents. We then measure the semantic correlation between each ARC question and the corresponding generated reflection.

We first obtain text embeddings using a pre-trained language model encoder to measure the semantic correlation between each ARC question and its corresponding reflection. Specifically, after the SLM agent attempts the ARC questions, we identify the incorrectly answered items and prompt both the SLM and LLM to generate reflections for those errors. We encode both texts for each question–reflection pair into fixed-size vector representations. We then compute the cosine similarity between the question embedding and its corresponding reflection embedding. This similarity score quantifies the degree of semantic alignment between the original question and the memory reflection generated, with higher scores indicating more substantial semantic overlap. This setup allows us to examine and contrast how reflection quality and abstraction differ between the SLM agents and the more capable LLM when addressing the same set of incorrect responses from the SLM agents.

Figure 3 presents the distribution of correlation scores between the question embeddings and the reflection embeddings. In the right plot, which depicts reflections generated by the SLM agent (DeepSeek-8B), we observe that both simple and complex prompts result in reflections with relatively high correlation to the original questions. This suggests that the SLM's limited reasoning capabilities constrain its ability to produce broad or deep critiques. The left plot shows that the LLM, when prompted with complex reflection instructions, generates reflections more correlated with the original question embeddings than simple reflections.

In the right plot of Figure 4, we observe that even when the LLM is prompted with the same simple instruction as the SLM agent, it produces reflections with lower correlation to the original ARC questions. This shows the LLM's ability to generate broader and more abstract analyses of the SLM agent's answers. The same is true for complex reflection prompts (see left plot). Overall, the comparisons in semantics show that the LLM is abstracting the incorrect answers generated by the SLM agent. This underlines the LLM's capacity for higher-level reflection compared to Deepseek-8B.

Analysis of cosine similarities in reflection embeddings with Phi-2. In the case of Phi-2, Figures 5 and 6 show the opposite facts. The SLM agent is able to produce reflections that are as abstract as the LLM model. However, when taking into account the accuracy scores discussed earlier, the simple and complex reflections generated by Phi-2 are not as effective as the ones provided by ChatGPT3.5.

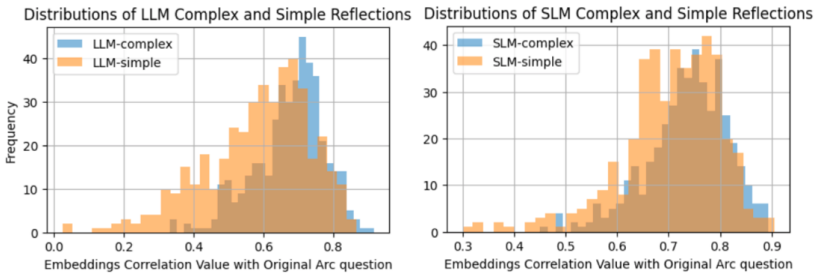

Fig. 3. Deepseek - Distribution of the cosine similarities in embeddings between language models (SLM or LLM) and reflection types (simple or complex) and original ARC questions.

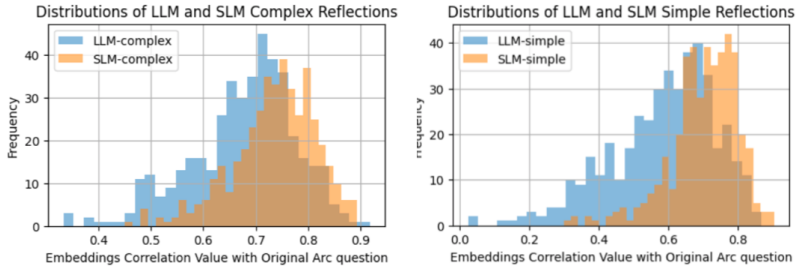

Fig. 4. Deepseek - Comparing the distribution of the cosine similarities in reflection embeddings generated by SLM and LLM models.

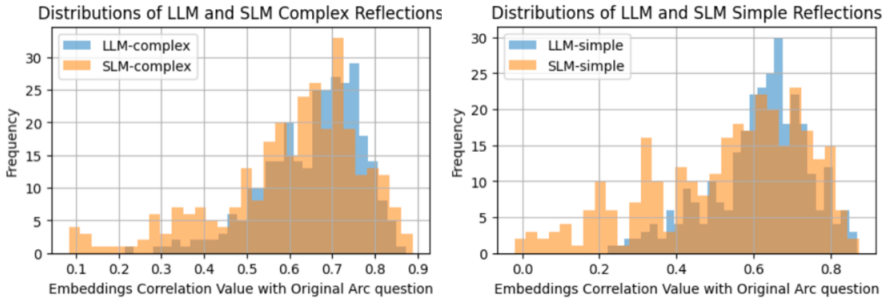

Fig. 5. Phi-2 - Comparing the cosine similarities in reflection embeddings generated by SLM and LLM models.

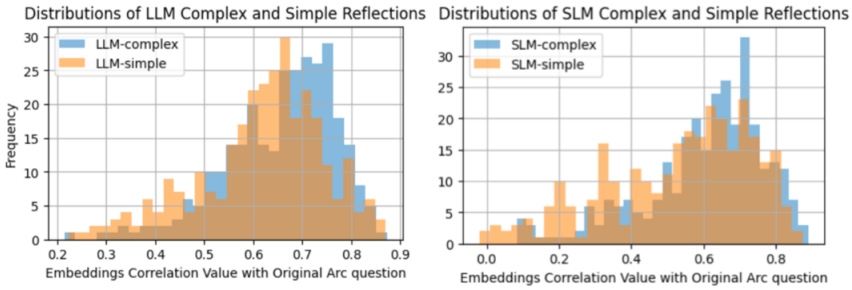

Fig. 6. Phi-2 - Cosine similarities in embeddings between language models (SLM or LLM) reflections and original ARC questions.

5.3 Reflections length analysis

Figure 7 presents the box plots of the length distributions by words and characters of the reflections generated by the LLM model and SLM agent Deepseek-8B. The comparison of reflection lengths across the four prompt types shows distinct differences in verbosity and response structure. The LLM's simple reflections are significantly shorter than the LLM's complex ones and those generated by the SLM agent.

Table 10 presents that among LLM reflections, the complex prompt yields longer responses (with a mean of 295.69 words per reflection) compared to the simple prompt (with a mean of about 65.91 words per reflection), reflecting the added instruction depth and elaboration expected from a complex reflection prompt. On the other hand, both SLM reflection types produce considerably longer responses, with SLM-complex reflections averaging over 385.05 and SLM-simple reflections at around 292.00.

We also note that the SLM's simple reflections are slightly longer than the LLM-complex ones, suggesting that the SLM tends to overgenerate in its reflective tasks. The high standard deviation in SLM responses compared to LLM responses, particularly in the simple variant, also indicates greater inconsistency in response length. We reach the same statements when measuring the reflection lengths by character (cf. Table 11). This shows that while the LLM model generates more concise reflections, the SLM agent generates more verbose and variable reflection output, which may not necessarily translate to better accuracy scores.

In the case of the SLM model Phi-2 (cf. Figure 8, and Tables 12 and 13), we observe similar results.

Reflection length is an important factor when prompting an agent in general, especially under limited context window constraints [7]. As we can see in Figure 7 and Figure 8, while the LLM's reflections remain concise and brief, the SLM agent reflections are consistently longer, with even the simple prompts almost reaching (Deepseek-8B) or exceeding (Phi-2) the length of the LLM's complex reflection outputs. The SLM agents' verbosity can quickly consume valuable

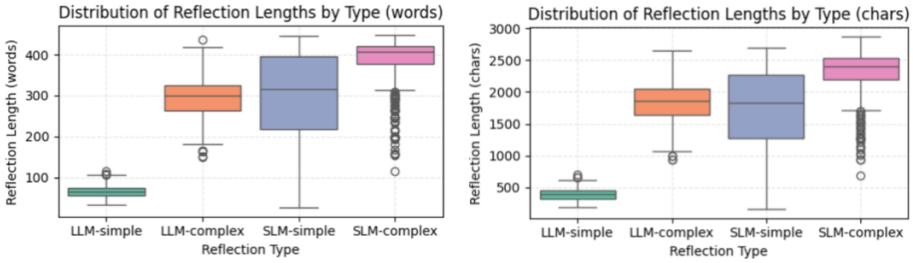

Fig. 7. Distributions of word and character lengths of reflections generated by LLM and SLM models and by type (simple versus complex), in the case of Deepseek-8B.

Table 10. Deepseek, reflection length statistical distribution by type (word count)

Reflection Type	Count	Mean	Std	Min	25%	50%	75%	Max
LLM-complex	428	295.69	48.77	150	263.00	299.0	326.00	437
LLM-simple	428	65.91	14.05	33	55.00	64.5	75.00	114
SLM-complex	428	385.05	60.25	115	377.50	407.5	421.25	449
SLM-simple	428	292.00	114.15	25	217.75	316.0	396.25	445

Table 11. Deepseek, reflection length statistical distribution by type (character count)

Reflection Type	Count	Mean	Std	Min	25%	50%	75%	Max
LLM-complex	428	1837.42	313.59	942	1634.75	1857.0	2049.50	2650
LLM-simple	428	398.62	86.18	188	329.00	390.5	453.00	705
SLM-complex	428	2298.28	364.05	684	2196.50	2400.5	2525.00	2874
SLM-simple	428	1712.41	663.06	155	1279.50	1828.0	2265.75	2696

context window space, reducing room for additional context, instructions, and memory. In addition, the high standard deviation in the SLM agents' outputs prevents predictability in prompt design. These findings underscore our external reflection approach as more effective for SLM agents' output improvements.

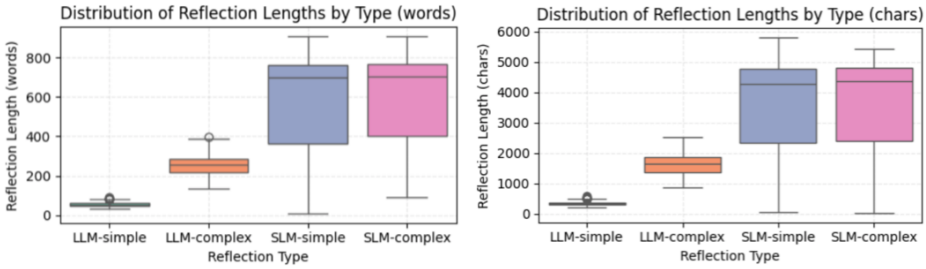

Fig. 8. Distributions of word and character lengths of reflections generated by LLM and SLM models and by type (simple versus complex), in the case of Phi-2

Table 12. Phi-2, reflection length statistical distribution by type (word count)

Reflection Type	Count	Mean	Std	Min	25%	50%	75%	Max
LLM-complex	301	255.22	50.86	134	218.0	257.0	287.0	399
LLM-simple	301	53.95	10.65	34	46.0	53.0	60.0	89
SLM-complex	299	589.38	224.93	92	401.0	702.0	769.0	910
SLM-simple	301	575.91	244.56	6	365.0	697.0	763.0	906

Table 13. Phi2, reflection length statistical distribution by type (character count)

Reflection Type	Count	Mean	Std	Min	25%	50%	75%	Max
LLM-complex	301	1615.42	338.32	866	1358.0	1637.0	1849.0	2511
LLM-simple	301	334.14	63.81	191	292.0	327.0	366.0	575
SLM-complex	301	3661.17	1455.82	3	2396.0	4362.0	4790.0	5438
SLM-simple	301	3568.57	1522.14	38	2339.0	4259.0	4771.0	5810

5.4 Reflections lexical diversity analysis

Figure 9 presents the box plots of lexical diversity for the two types of reflections provided by SLM agents and the LLM model. The plots show that the memory

reflections produced by LLM present a higher lexical diversity than those generated by both Deepseek-8B and Phi-2. This indicates a broader and more varied vocabulary use in the case of ChatGPT3.5. Among the LLM-generated reflections, complex reflections contain less diverse language than simple ones, despite the deeper reasoning evaluations instructed in the reflection prompts. Shorter, simpler reflections than complex ones can explain this. In contrast, while longer in length, SLM reflections show lower TTR scores, suggesting a tendency to repeat phrasing.

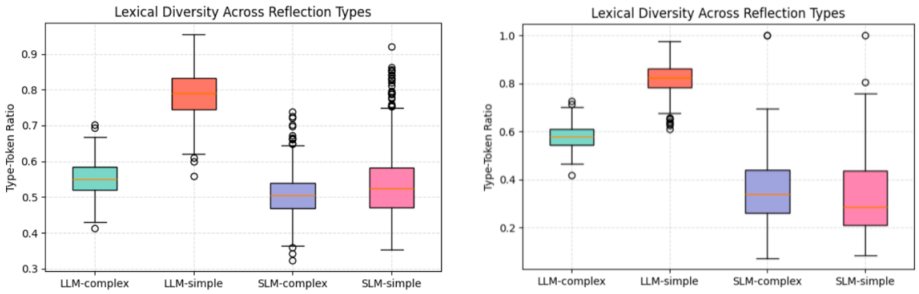

Fig. 9. Lexical diversity distributions of reflections generated by LLM and SLM agents and by type (simple versus complex). Left plot for Deepsek-8B and right plot for Phi-2.

6 Discussion

In our approach, rather than relying on the SLM to perform its own self-assessment, we introduced a mechanism in which the LLM evaluates the SLM's responses in light of user (or environment) feedback and produces a structured reflection. These reflections are stored in a reflection memory module, which serves as a source of corrective knowledge accumulated from the SLM's past performance. This external memory is then used to inform and improve the SLM's future outputs, enabling better alignment with user expectations without increasing inference-time computational costs.

Our experimental results demonstrated that delegating the reflection process to an LLM significantly improves the effectiveness of SLM agents compared to the baseline approach, where the SLM self-reflects. While the baseline relies on the limited capacity of the SLM to analyze and revise its own outputs, our method leverages the superior reasoning and evaluative capabilities of the LLM to generate higher-quality reflections. These externally generated reflections, when stored in the reflection memory and reintroduced during subsequent interactions, provide more accurate, targeted, and informative feedback than what the SLM is capable of producing alone.

Reflection length emerges as an relevant factor in this setup, particularly under limited context window size constraints. Our analysis demonstrated that SLM-generated reflections are considerably longer and more variable than those produced by the LLM model. Even simple SLM reflections exceed the length of complex LLM ones (e.g., the SLM agent Phi-2 in our experiments), suggesting that SLMs tend to overgenerate, possibly due to limited summarization control. This verbosity can consume the prompt space and reduce the room for additional memory or context, potentially undermining efficiency. In contrast, LLM reflections remained more concise and consistent, making them more informative and practical for integration in multi-step or memory-augmented architectures. These findings highlighted the dual advantage of LLM reflections: improved content quality and greater prompt efficiency.

Future work aims to explore methods for synthesizing the more verbose complex reflections generated by the SLM agent using a LLM. This compression step will serve to reduce reflection lengths while keeping the wordings used by the SLM, mitigate the risk of the SLM being stuck in error loops, and improve efficiency in memory usage and retrieval for downstream prompting. We also plan to investigate alternative prompt designs, employ other LLMs in our studies and apply our solution to different tasks and domains. This will also allow us to discover potential limitations in our approach and conditions under which external reflections may not improve SLM agents' effectiveness.

7 Conclusion

Ensuring that reflections are concise yet meaningful regarding agents' reflection memory is essential for effective context reuse in future prompts. However, SLMs present constrained architectures for performing complex reasoning and self-evaluation. This study presented a novel reflection-based framework designed to enhance the effectiveness of SLM-based agents through externalized introspection. Our approach explored using LLMs to generate reflective feedback on the SLM's outputs. We found that LLM-generated reflections are better quality and significantly more concise and consistent than SLM-generated ones. Our experimental evaluations conducted on the ARC challenge dataset, using the SLMs Deepseek (8B) and Phi-2 (2.7B), demonstrated the effectiveness of using the LLM (ChatGPT3.5) for reflecting on the SLM's incorrect answers. For instance, when prompted with external reflections, Deepseek-8B increased its accuracy score from 62.78% to 81.38% compared to 77.70% with its self-reflections. Our results showed that the LLM's reflections decrease the risk of the SLM agents outputting the same incorrect answers and being stuck in an error loop. Our study demonstrated that using LLMs for reflection generation enhances the effectiveness of SLM agents and contributes to a more scalable and memory-efficient solution. The obtained findings have implications for context window management and system efficiency, as shorter reflections reduce memory usage and faster retrieval when incorporating relevant reflection memory into new prompts. Future studies aim to investigate a cost–benefit analysis to bet-

ter quantify the trade-offs between the improved accuracy gains from external reflections and the additional computational cost of invoking LLMs.

Acknowledgments. This study was sponsored by Petróleo Brasileiro S.A. (PETROBRAS) within the project "*Application of Large Language Models (LLMs) for online monitoring of industrial processes*" conducted in partnership with the University of Campinas [01-P-34480/2024 - 62208].

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Gallegos, I.O., Rossi, R.A., Barrow, J., Tanjim, M.M., Kim, S., Dernoncourt, F., Yu, T., Zhang, R., Ahmed, N.K.: Bias and fairness in large language models: A survey (2024), https://arxiv.org/abs/2309.00770
2. Guo, J., Wang, M., Yin, H., Song, B., Chi, Y., Yu, F.R., Yuen, C.: Large language models and artificial intelligence generated content technologies meet communication networks (2024), https://arxiv.org/abs/2411.06193
3. Kagaya, T., Yuan, T.J., Lou, Y., Karlekar, J., Pranata, S., Kinose, A., Oguri, K., Wick, F., You, Y.: Rap: Retrieval-augmented planning with contextual memory for multimodal llm agents (2024), https://arxiv.org/abs/2402.03610
4. Kang, M., Jeong, J., Lee, S., Cho, J., Hwang, S.J.: Distilling llm agent into small models with retrieval and code tools (2025), https://arxiv.org/abs/2505.17612
5. Li, Y., Yang, C., Ettinger, A.: When hindsight is not 20/20: Testing limits on reflective thinking in large language models (2024), https://arxiv.org/abs/2404.09129
6. Li, Y., Du, M., Song, R., Wang, X., Wang, Y.: A survey on fairness in large language models (2024), https://arxiv.org/abs/2308.10149
7. Liang, X., Tao, M., Xia, Y., Shi, T., Wang, J., Yang, J.: Self-evolving agents with reflective and memory-augmented abilities (2024), https://arxiv.org/abs/2409.00872
8. Lippmann, P., Spaan, M.T.J., Yang, J.: Positive experience reflection for agents in interactive text environments (2024), https://arxiv.org/abs/2411.02223
9. Lu, J., An, S., Lin, M., Pergola, G., He, Y., Yin, D., Sun, X., Wu, Y.: Memochat: Tuning llms to use memos for consistent long-range open-domain conversation (2023), https://arxiv.org/abs/2308.08239
10. M, J., P, V.S., Kryvinska, N.: Exploring the chatbot usage intention-a mediating role of chatbot initial trust. Heliyon **10**(12), e33028 (2024). https://doi.org/https://doi.org/10.1016/j.heliyon.2024.e33028, https://www.sciencedirect.com/science/article/pii/S2405844024090595
11. Piao, S., Park, S.: Tinythinker: Distilling reasoning through coarse-to-fine knowledge internalization with self-reflection (2025), https://arxiv.org/abs/2412.08024
12. Renze, M., Guven, E.: Self-reflection in llm agents: Effects on problem-solving performance (2024), https://arxiv.org/abs/2405.06682
13. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning (2023), https://arxiv.org/abs/2303.11366

14. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W.X., Wei, Z., Wen, J.: A survey on large language model based autonomous agents. Frontiers of Computer Science **18**(6) (Mar 2024). https://doi.org/10.1007/s11704-024-40231-1, http://dx.doi.org/10.1007/s11704-024-40231-1
15. Wu, Y., Xu, G., Dongchen, Z.: proposalself-reflection like humans, editable-LLM (e-LLM) is all you need (2024), https://openreview.net/forum?id=rk6PJlu562, under review
16. Zhang, Z., Bo, X., Ma, C., Li, R., Chen, X., Dai, Q., Zhu, J., Dong, Z., Wen, J.R.: A survey on the memory mechanism of large language model based agents (2024), https://arxiv.org/abs/2404.13501
17. Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.J., Huang, G.: Expel: Llm agents are experiential learners (2024), https://arxiv.org/abs/2308.10144